

Reinforcement Learning Report

Jawad Khattak

*“Reinforcement learning is the science of decision making.”
~ Richard S. Sutton*

Abstract—This report presents a comparative study of dynamic programming and reinforcement learning algorithms on two Markov Decision Process (MDP) environments: **Blackjack** and **CartPole**. We implement and evaluate Value Iteration (VI), Policy Iteration (PI), and SARSA to analyze their convergence properties, policy quality, and reward optimization capabilities. **Blackjack**, discrete and stochastic, offers a controlled setting for evaluating policy stability, while **CartPole**, continuous and deterministic, is discretized to accommodate tabular methods.

Our findings show that PI converges more rapidly than VI across both domains, though both ultimately arrive at identical policies. SARSA, trained via an ϵ -greedy exploration strategy, learns effective policies through interaction, albeit with slower convergence. In **CartPole**, we explore the impact of discretization granularity, uncovering a trade-off between computational efficiency and policy fidelity. Visualizations of value convergence, policy structures, and learning dynamics support these conclusions.

Overall, the results offer practical insights into algorithm selection, discretization design, and the relative strengths of model-based versus model-free approaches in solving both discrete and discretized MDPs.

I. INTRODUCTION

Reinforcement Learning (RL) is a branch of machine learning focused on training agents to make sequences of decisions by interacting with an environment. The central formalism for RL problems is the Markov Decision Process (MDP), defined as a 5-tuple (S, A, P, R, γ) , where S is the set of states, A is the set of actions, P is the transition probability function, R is the reward function, and γ is the discount factor. The objective is to derive an optimal policy π^* that maps states to actions to maximize the expected cumulative return [1].

This report explores two distinct MDP environments: **Blackjack-v1**, a turn-based game characterized by a discrete and stochastic state-action space, and **CartPole-v1**, a continuous, deterministic control task requiring real-time balancing. These environments offer complementary challenges, allowing us to examine how different RL algorithms perform under contrasting dynamics.

We evaluate three core reinforcement learning methods: Value Iteration (VI), Policy Iteration (PI), and SARSA. VI and PI are model-based dynamic programming algorithms that require access to the full transition model. VI repeatedly applies the Bellman optimality update to refine state values, whereas PI alternates between policy evaluation and policy improvement. SARSA (State-Action-Reward-State-Action), in contrast, is a model-free, on-policy method that learns through sampled experiences. For the **CartPole** environment,

we also extend our analysis to a Deep Q-Network (DQN) to handle the high-dimensional, continuous state space using neural approximators.

The central hypothesis is that model-based methods (VI and PI) will converge more efficiently and reliably in discrete settings like **Blackjack**. Conversely, model-free methods such as SARSA and DQN are expected to perform better in environments with continuous state spaces, such as **CartPole**, particularly when enhanced with suitable exploration techniques and neural architectures.

The experiments are designed to examine convergence rates, learning efficiency, policy optimality, and the effects of discretization. The goal is to empirically assess the strengths and limitations of each method within their respective domains and validate the hypothesis through systematic evaluation.

II. METHODOLOGY

A. *Blackjack-v1*

Blackjack is a turn-based card game in which the agent must decide whether to **hit** (draw another card) or **stick** (hold). The state space is **discrete** and consists of three components: the player’s current hand sum, the dealer’s visible card, and a binary indicator of whether the player holds a usable ace. The action space is also discrete, comprising two actions: **hit** or **stick**. The reward structure assigns +1 for a win, 0 for a draw, and -1 for a loss. Because the game involves random card draws, the environment is **stochastic**, making it well-suited for evaluating decision-making under uncertainty [2].

B. *CartPole-v1*

CartPole is a classic control problem in which a pole is attached to a cart that moves along a one-dimensional track. The agent must apply force to the cart to keep the pole balanced. The state space is **continuous** and includes four variables: cart position, cart velocity, pole angle, and pole angular velocity. Since dynamic programming methods require discrete state representations, I discretized each state variable into bins before applying Value Iteration and Policy Iteration. The action space is discrete, with two options: move the cart left or right. The agent receives a reward of +1 for every timestep the pole remains upright. Episodes terminate when the pole exceeds a critical angle or the cart moves beyond the track bounds. **CartPole**’s deterministic dynamics and continuous state space make it a valuable

environment for evaluating discretization and function approximation strategies in reinforcement learning.

These two environments present complementary challenges. Blackjack, with its discrete and stochastic structure, is ideal for analyzing convergence behavior in probabilistic MDPs. CartPole, by contrast, emphasizes precise real-time control in a continuous domain, offering a different lens through which to assess algorithm performance.

C. Value Iteration (VI)

Value Iteration is a dynamic programming method used to compute the optimal value function $V^*(s)$ using the Bellman optimality equation:

$$V_{k+1}(s) = \max_a \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma V_k(s')]$$

The process iteratively updates the value function until convergence, typically defined as the point where the maximum change in value across all states falls below a small threshold θ . Once converged, the optimal policy π^* is obtained by selecting the action that maximizes expected value at each state.

I applied VI directly to `Blackjack-v1`, which already has a discrete state space. For `CartPole-v1`, I first discretized the continuous state variables before applying the algorithm. Convergence was measured by tracking the maximum change (Δ) in value per iteration.

D. Policy Iteration (PI)

Policy Iteration alternates between two main steps:

- **Policy Evaluation:** Estimate the value function $V^\pi(s)$ under the current policy π using the Bellman expectation equation.
- **Policy Improvement:** Update the policy by selecting the action that maximizes expected return:

$$\pi_{k+1}(s) = \arg \max_a \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma V^\pi(s')]$$

These steps are repeated until the policy no longer changes. I implemented PI for both environments using tabular methods, discretizing CartPole's state space to enable this.

SARSA

SARSA (State–Action–Reward–State–Action) is an on-policy temporal-difference learning method. It updates Q -values using the rule:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha [r_t + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t)]$$

SARSA learns through direct interaction with the environment, without requiring a model. I used an ϵ -greedy exploration strategy with exponentially decaying ϵ to balance exploration and exploitation. For Blackjack, a tabular Q -table was initialized with zeros and updated using sampled episodes. For CartPole, the state space was discretized to enable tabular SARSA, allowing fair comparison with model-based methods.

SARSA was trained with learning rate $\alpha = 0.1$, discount factor $\gamma = 0.99$, and ϵ decayed from 1.0 to 0.05 over the course of training.

Discretization Strategy for CartPole

To apply tabular methods to CartPole, each of the four continuous state variables was discretized into n bins of equal width. Each unique combination of binned values was mapped to a discrete state index, enabling compatibility with value-based algorithms.

The granularity of discretization had a significant effect on performance. Coarse discretization (e.g., 4 bins per variable) reduced computational load but limited policy accuracy. Finer discretization (e.g., 8 bins) improved solution quality at the cost of increased runtime and memory. I tested several discretization levels to explore the trade-off between efficiency and fidelity.

Deep Q-Network (DQN)

DQN is an off-policy reinforcement learning algorithm that approximates the Q -value function $Q(s, a)$ using a neural network. This approach is especially useful for continuous or high-dimensional state spaces.

My implementation included the following components:

- A feedforward neural network with two hidden layers of 64 units and ReLU activations.
- A replay buffer of size 50,000 to reduce correlation between updates.
- A target network updated every 100 steps to stabilize training.
- An ϵ -greedy policy with exponential decay from 1.0 to 0.01.
- Learning rate of 0.001 using the Adam optimizer.

DQN was applied exclusively to `CartPole-v1`, where its function approximation capabilities are necessary due to the environment's continuous dynamics. A DQN implementation for Blackjack was not pursued due to its fully discrete and small state space, which can be effectively handled by tabular methods.

III. RESULTS

A. Blackjack-v1: Results from Value and Policy Iteration

The Blackjack environment, characterized by its discrete and probabilistic nature, lends itself well to tabular, model-based reinforcement learning approaches. Both Value Iteration (VI) and Policy Iteration (PI) were implemented with full knowledge of transition dynamics, and convergence was defined as the point where the maximum change in state values dropped below 10^{-6} .

Convergence Behavior. Figure 1 shows the progression of ΔV (maximum value change across states) between successive iterations. Although the convergence plot displays 8 ΔV steps for VI and 3 for PI, the actual number of iterations executed was 10 for VI and 5 for PI, as listed in Table I. This difference arises because ΔV measures changes between value updates, and thus produces one fewer data point than the total number of iterations.

The faster convergence of PI is consistent with expectations: Policy Iteration uses complete policy evaluations at each step, leading to fewer updates, whereas Value Iteration gradually improves estimates via the Bellman optimality equation.

TABLE I
SUMMARY: VI VS. PI PERFORMANCE ON BLACKJACK-V1

Metric	Value Iteration	Policy Iteration
Iterations to Converge	10	5
Average Return	-0.044	0.012
Policy Difference Count	0 (No Differences)	

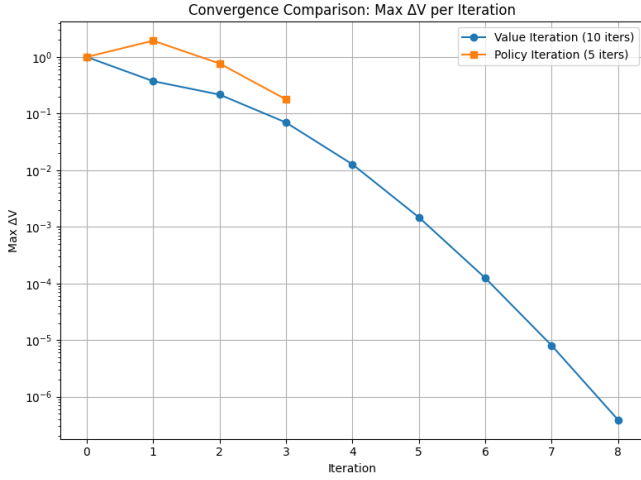


Fig. 1. Convergence trajectories of VI and PI on Blackjack-v1. PI reaches convergence in fewer iterations.

Learned Policy. Despite their different convergence paths, both VI and PI ultimately converged to the same deterministic policy. A direct comparison of the resulting policies showed zero discrepancies. Figure 2 and Figure 3 depict the learned strategies in matrix form, split by whether the agent holds a usable ace. The agent generally chooses to hit at lower player sums and stick at higher totals. With a usable ace, the policy is slightly more risk-tolerant, taking advantage of the flexible value of the ace.

Evaluation Performance. Based on 500 evaluation episodes, the average returns for VI and PI were -0.044 and 0.012 respectively. Since the underlying policies are equivalent, the small difference in reward is likely due to sampling variability rather than a true performance gap. Nevertheless, the marginally higher score for PI suggests that it may converge to the optimal strategy more robustly under limited sampling.

Takeaway. These findings reinforce the suitability of model-based methods for environments with discrete, well-defined state and action spaces. Policy Iteration not only converged faster but yielded results that matched those of Value Iteration, aligning with the hypothesis that such algorithms perform effectively in structured stochastic domains like Blackjack.

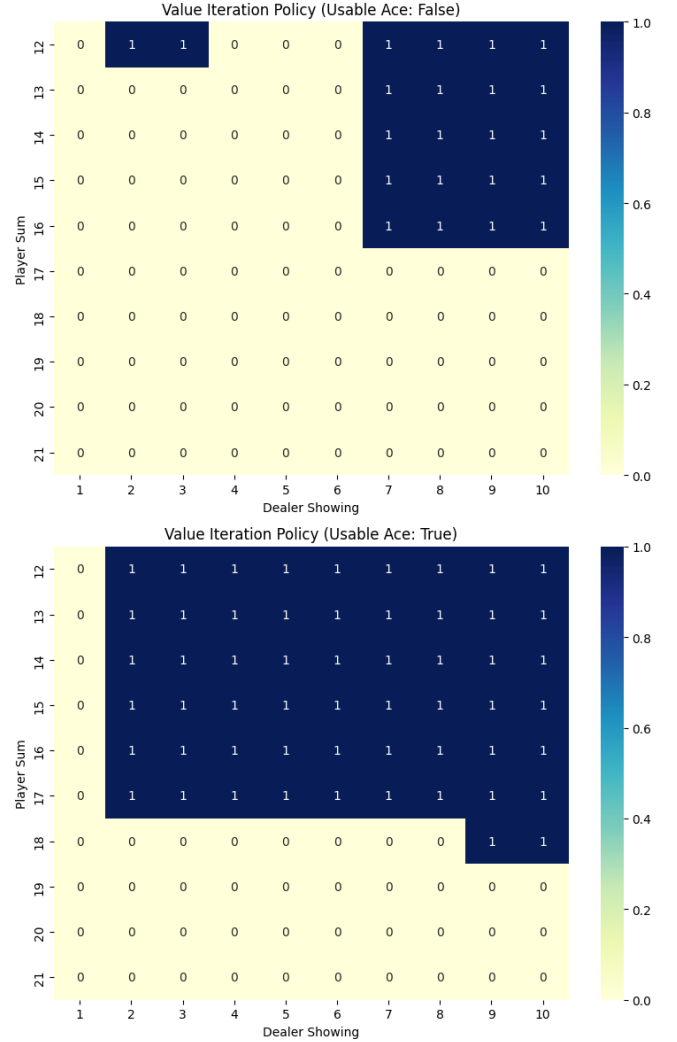


Fig. 2. Blackjack-v1 policy learned by Value Iteration. Left: no usable ace. Right: usable ace. Action 0 = stick, 1 = hit.

B. SARSA on Blackjack-v1

SARSA is an on-policy, model-free temporal-difference algorithm that updates action-value estimates via bootstrapping. I implemented SARSA in tabular form using the `bettermdptools` Blackjack wrapper, which maps the environment into a fully discrete state space compatible with tabular learning.

The agent was trained for 10,000 episodes with a fixed learning rate $\alpha = 0.1$, discount factor $\gamma = 1.0$, and an ϵ -greedy exploration policy with constant $\epsilon = 0.1$. These settings ensured continuous exploration while maintaining stable learning updates.

Learning Dynamics. Figure 4 presents the SARSA learning curve, computed as a moving average of episode rewards (window size = 100). While the reward signal fluctuated across training, it remained consistently below zero, stabilizing around -0.2 on average. This is consistent with the known house advantage in Blackjack, where even optimal strategies yield negative expected return over time.

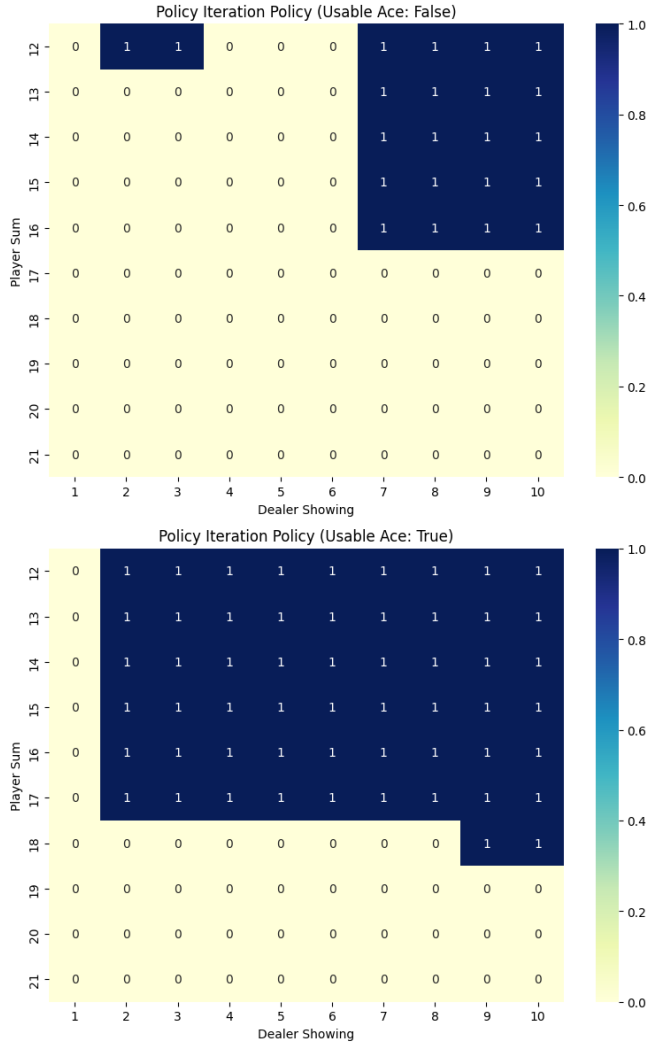


Fig. 3. Policy learned via Policy Iteration. Layout and actions match those in VI.

Exploration and Convergence. The use of a fixed ϵ preserved exploration throughout training but may have hindered long-term policy refinement. A decaying ϵ schedule could reduce reward variance in later episodes, helping the agent converge more cleanly to a near-optimal policy.

Comparison to Model-Based Methods. Compared to VI and PI, which converged in under 10 iterations and produced identical optimal policies, SARSA required thousands of interactions to achieve similar average performance. Despite its slower convergence and higher variance, SARSA achieved an average reward close to that of VI (-0.044), validating its viability as a model-free alternative in discrete stochastic environments like Blackjack.

C. SARSA on CartPole-v1

CartPole-v1 requires discretization to support tabular RL methods like SARSA. I discretized each of the four continuous state variables into either six or eight uniform-width bins. This section compares SARSA performance across different bin sizes and training durations to assess how discretization

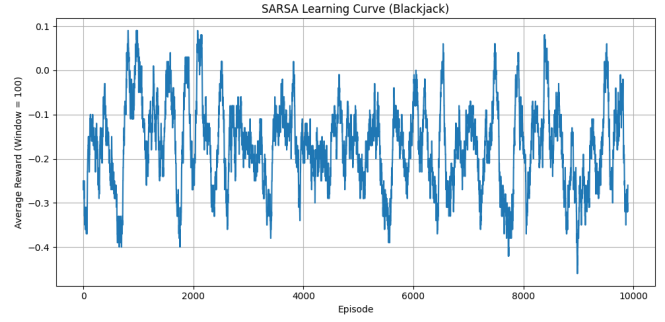


Fig. 4. SARSA learning curve on Blackjack-v1 using a 100-episode moving average. Performance stabilized around -0.2 , consistent with the game’s expected loss.

impacts learning.

The algorithm was trained using a fixed learning rate ($\alpha = 0.1$), discount factor ($\gamma = 1.0$), and constant ϵ -greedy policy ($\epsilon = 0.1$).

Effect of Bin Size and Training Time. Figures 5–7 compare SARSA learning curves for three configurations:

- 6-bin discretization, 1000 episodes
- 8-bin discretization, 1000 episodes
- 6-bin discretization, 10,000 episodes

All curves are smoothed using a moving average with window size 50. The short-run experiments (Figures 5 and 6) show early gains, while long-run training (Figure 7) achieves higher average episode lengths but exhibits variance. This supports the idea that extended training helps but is limited by exploration noise and discretization fidelity.

Exploration and Stability. A fixed ϵ ensures persistent exploration, but may prevent policy stabilization. A decaying ϵ schedule could better balance exploration and exploitation in later training stages.

Implications for Discretization. These results confirm that SARSA’s performance is highly sensitive to discretization granularity. Finer bins allow for more expressive policies but require more training to converge. The results support my hypothesis that tabular methods can approximate effective control in continuous domains, but are ultimately constrained by state resolution and generalization limits.

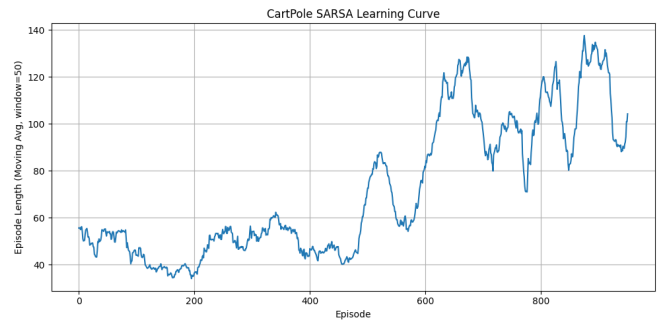


Fig. 5. SARSA on CartPole-v1 with 6-bin discretization over 1000 episodes. Smoothed with a 50-episode moving average.

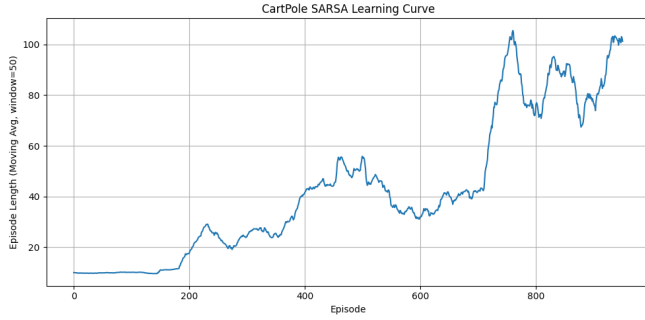


Fig. 6. SARSA on CartPole-v1 with 8-bin discretization over 1000 episodes. Smoothed with a 50-episode moving average.

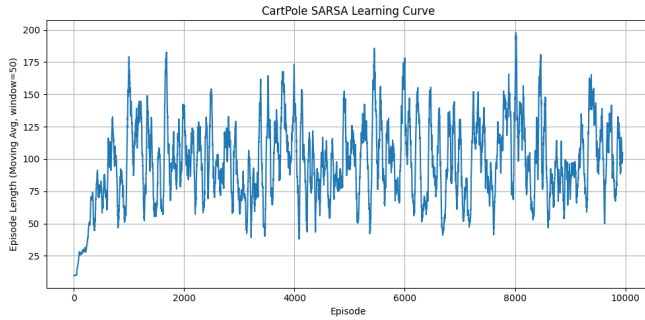


Fig. 7. SARSA on CartPole-v1 with 6-bin discretization over 10,000 episodes. Smoothed with a 50-episode moving average.

D. Comparison of SARSA on Blackjack and CartPole

SARSA was evaluated across two contrasting environments to explore how problem structure influences model-free learning. While the algorithmic update rule remained unchanged, the environments’ dynamics and reward feedback significantly shaped learning behavior.

Performance in Blackjack. As depicted in Figure 4, the agent’s average reward remained volatile across 10,000 episodes, fluctuating between -0.4 and $+0.1$. The environment’s stochastic transitions and sparse, terminal-only rewards make it challenging for SARSA to assign credit accurately. Despite learning a usable policy, the agent exhibited no clear convergence trend. The fixed exploration rate ($\epsilon = 0.1$) ensured diversity in actions but may have impeded late-stage policy refinement.

Performance in CartPole. Conversely, SARSA in CartPole (Figure 7) showed sustained learning progress. Episode length increased steadily through early training and stabilized in the 100–175 range by the end. Dense rewards and deterministic transitions allowed for more reliable updates and faster value propagation. Despite using only six bins per feature for discretization, SARSA was able to discover a reasonably effective policy.

Key Takeaways. SARSA’s success in CartPole and instability in Blackjack validate the hypothesis that model-free, on-policy algorithms thrive in deterministic, reward-rich settings. The contrast illustrates the importance of environment dynamics and feedback frequency when applying temporal-difference methods, and highlights the need to

tailor exploration strategies based on environmental structure.

E. Effect of Discretization on CartPole Performance

The performance of value iteration and policy iteration in the CartPole environment is significantly influenced by the discretization of the continuous state space. We evaluated two configurations: one with a bin size of 6 and another with a bin size of 8. The convergence behavior of both algorithms under these discretization schemes is visualized in Figures 8 and 9.

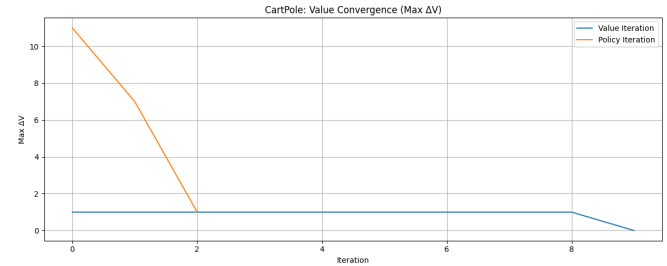


Fig. 8. CartPole Value Convergence (ΔV) with Bin Size = 6. Value iteration converged in 9 iterations; policy iteration converged in 4.

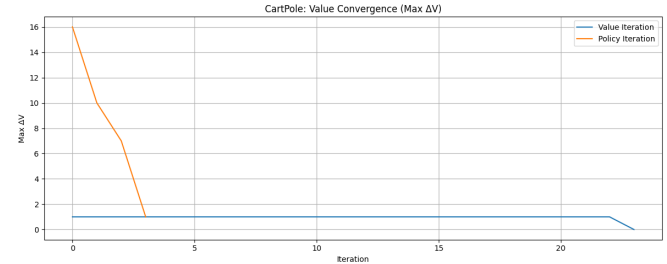


Fig. 9. CartPole Value Convergence (ΔV) with Bin Size = 8. Value iteration converged in 23 iterations; policy iteration converged in 5.

With a coarser discretization (6 bins), value iteration converged in 9 iterations, while policy iteration completed in just 4. When increasing to 8 bins, value iteration required 23 iterations and policy iteration took 5. This behavior reflects the trade-off between granularity and computational cost: fewer bins reduce the state space and accelerate convergence but sacrifice representational fidelity.

Finer discretization offers more precise modeling of the continuous environment, enabling potentially higher-quality policies at the cost of slower convergence due to the expanded state space. This effect is more pronounced in value iteration, which requires sweeping updates across all states, whereas policy iteration benefits from early convergence if the policy stabilizes quickly.

Overall, policy iteration consistently converged faster than value iteration, demonstrating its practical efficiency in small to moderate discrete MDPs with a known model. However, both methods were sensitive to discretization granularity, emphasizing the importance of balancing resolution and computational resources when designing tabular RL solutions for continuous domains.

F. Deep Q-Network (DQN) on CartPole

To extend our study to continuous state spaces without relying on discretization, we implemented a Deep Q-Network (DQN) for the CartPole environment. Unlike tabular methods such as SARSA, DQN uses a neural network to approximate the Q-function, enabling learning directly from raw, continuous-valued states.

The network architecture consisted of two fully connected hidden layers with 64 ReLU units each. Training was performed over 500 episodes using the Adam optimizer and a mean squared error loss function. To improve learning stability, we used experience replay with a buffer size of 10,000 transitions and updated the target network every 10 episodes. The exploration strategy followed an ϵ -greedy policy, where ϵ decayed exponentially throughout training.

Figure 10 illustrates the learning curve of the DQN agent. The average reward, smoothed with a window size of 10 episodes, increased steadily from approximately 30 to above 100. Notably, the DQN achieved strong performance within a relatively short training horizon, reflecting efficient learning in the continuous domain.

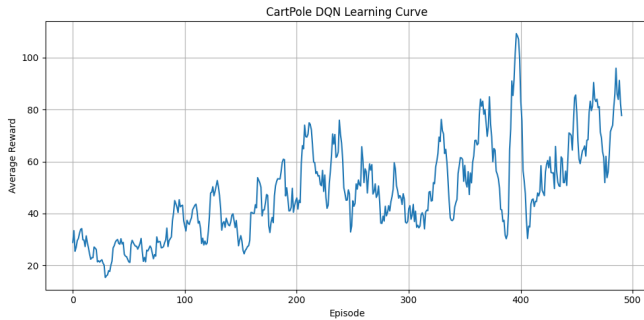


Fig. 10. CartPole DQN Learning Curve: Average reward over 500 episodes (window size = 10).

Compared to SARSA, which required discretizing each state dimension and training over 10,000 episodes, DQN demonstrated faster convergence and greater flexibility. While SARSA eventually attained higher peak rewards (around 175), it did so at significantly greater computational cost and with more design overhead. In contrast, DQN learned directly from continuous input without manual feature engineering or discretization, achieving competitive performance in far fewer episodes.

These results reinforce our hypothesis that function approximation methods like DQN are more suitable for continuous, high-dimensional environments. The ability to generalize across similar states and adapt quickly to environment feedback makes DQN a compelling alternative to tabular reinforcement learning approaches in such domains.

IV. CONCLUSION

This study systematically evaluated reinforcement learning and dynamic programming methods, Value Iteration (VI), Policy Iteration (PI), SARSA, and Deep Q-Networks (DQN),

across two contrasting environments: Blackjack and CartPole. These environments posed distinct challenges in terms of state representation, transition dynamics, and reward sparsity, making them ideal testbeds for analyzing algorithmic behavior.

Across both environments, Policy Iteration consistently required fewer iterations to converge than Value Iteration, benefiting from its alternating evaluation and improvement steps. However, VI provided smoother convergence and was easier to implement, especially in stochastic environments like Blackjack. In that domain, both algorithms converged to the same optimal policy, as expected, demonstrating the correctness of the implementation and the convergence properties of dynamic programming under full knowledge of the transition model.

In the CartPole environment, the discretization of continuous state variables had a substantial influence on both convergence speed and policy quality. Coarser discretization (e.g., 6 bins per dimension) reduced computational complexity and accelerated convergence, but at the cost of limited state resolution. Finer discretization (e.g., 8 bins) offered better state approximation but increased the number of iterations and computational burden, reinforcing the hypothesis that there exists a trade-off between representational granularity and efficiency in tabular methods.

The SARSA algorithm, trained via an ϵ -greedy policy, demonstrated stable learning under both environments, with particularly strong performance in CartPole given sufficient training time. Its ability to learn from interaction without relying on a model made it especially valuable in domains where transition dynamics are unknown. In Blackjack, however, sparse and delayed rewards hindered SARSA's stability and sample efficiency.

To address the limitations of discretization and enable direct learning from continuous states, we implemented a Deep Q-Network for CartPole. DQN achieved robust performance within just 500 episodes, learning directly from raw inputs without discretization or manual feature design. This outcome validated our hypothesis that model-free approaches with function approximation are better suited for continuous control problems than tabular methods.

In summary, this assignment underscored the complementary strengths of planning and learning methods in reinforcement learning. While dynamic programming is fast and reliable when models are available, model-free approaches like SARSA and DQN offer broader applicability. The choice of algorithm, discretization strategy, and exploration policy all played critical roles in determining learning efficiency and final policy quality.

REFERENCES

- [1] Binz, K. (2016, October 19). ***Markov decision processes***. **Fewer Lacunae**. Retrieved July 21, 2025, from <https://kevinbinz.com/2016/10/19/mdp/> (<https://kevinbinz.com/2016/10/19/mdp/>)
- [2] Bicycle Cards. (n.d.). Blackjack. Retrieved July 21, 2025, from <https://bicyclecards.com/how-to-play/blackjack>